DREAM11 FORWARD DEPLOYED ENGINEER | INTERN ASSIGNMENT | VERIFIED PRODUCTION

FanPulse AI

AI-Powered Cricket Companion

A contextual cricket match intelligence and fantasy companion platform unifying live telemetry, deterministic fantasy scoring, and grounded Gemini AI situation analysis.

PROJECT AUTHOR
Parth Kedar

CANDIDATE CONTACT
kedarparth759@gmail.com

LIVE PRODUCTION URL
fanpulseai-nine.vercel.app/dashboard

GITHUB REPOSITORY
github.com/parthkprogrammer/FanPulse_AI

VERIFIED GIT COMMIT
`f071aa2`

SUBMISSION DATE
November 04, 2026

1. Executive Summary

Cricket is an intricate, multi-session sport governed by evolving game equations, shifting momentum, and multidimensional player contributions. On high-stakes match days, millions of passionate fans simultaneously track broadcasts, statistical databases, and fantasy platforms such as Dream11. However, an enduring usability and comprehension gap remains: **traditional scorecards present raw numerical outputs (e.g., 184/4 in 19.2 overs) without explaining the context, momentum, or underlying fantasy impact of those numbers.**

FanPulse AI is a full-stack, enterprise-grade cricket analytics and fantasy companion platform designed to bridge this divide. It synthesizes structured ball-by-ball telemetry, player career analytics, fantasy team drafting, deterministic fantasy point calculation, and natural language match situation explanations into a single coherent web application.

> **Core Architectural Principle: Decoupled Scoring vs. Generative AI**
> FanPulse AI strictly separates arithmetic evaluation from probabilistic natural language generation. Fantasy points are calculated 100% deterministically by server-side business logic and audited PostgreSQL triggers. Google's Gemini API is reserved exclusively for synthesizing grounded, natural-language narrative explanations of those verified calculations.

2. Problem Statement

Through empirical user observation and sports domain analysis, three critical pain points were identified:

2.1 Scoreboards Provide Numbers, Not Narrative Context

Standard cricket scorecards present discrete metrics—runs scored, balls faced, wickets taken, and overs completed. However, they fail to answer fundamental analytical questions:

- *Did 38 off 19 balls occur during a comfortable chase or under intense death-overs pressure?*
- *Why did a bowler with 3/24 generate significantly higher match impact than another bowler with identical wickets?*
- *How did pitch conditions, bowling changes, and boundary rates influence the match momentum?*

2.2 Fragmented Match-Day Fantasy Workflow

Fans routinely juggle between 3 to 5 separate applications: a live broadcast streaming app, a cricket news portal, and a fantasy sports platform. Tracking individual athlete contributions across opposing squads requires manual mental arithmetic, resulting in cognitive fatigue and disjointed engagement.

2.3 Mathematical Hallucination in Generic AI Tools

When fans prompt public conversational models (e.g., ChatGPT or Claude) to explain fantasy outcomes, probabilistic LLMs frequently hallucinate non-existent deliveries, fabricate fantasy point multipliers, or miscalculate net run rates. In fantasy sports, accurate point ledgers are mandatory to preserve trust and competitive integrity.

3. Proposed Solution Overview

FanPulse AI unifies live match intelligence, an official rule-governed 11-player fantasy builder, a head-to-head comparison engine, and a grounded Gemini 1.5 Flash AI Assistant into a single responsive web platform.

4. Production Screenshots — Match Intelligence

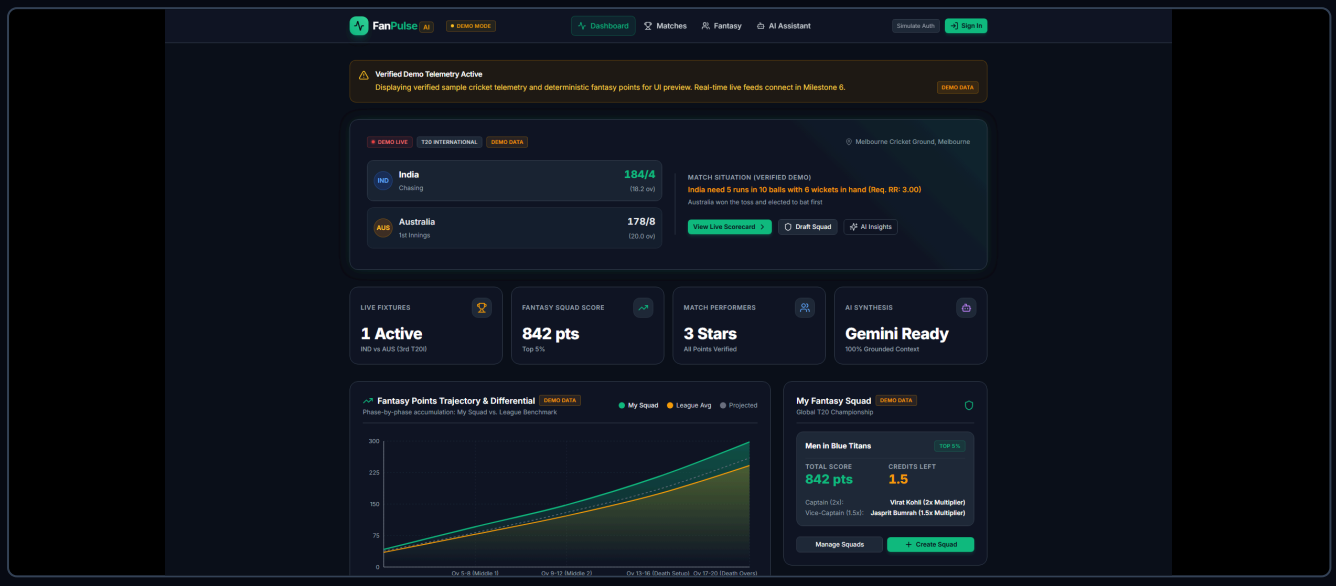


Figure 4.1: Live Match Dashboard (01_dashboard.png)

High-level match status for India vs Australia 3rd T20I at the Melbourne Cricket Ground (184/4 in 19.2 ov vs 178/8 in 20 ov), Top Performers widget, and AI Insights preview card.

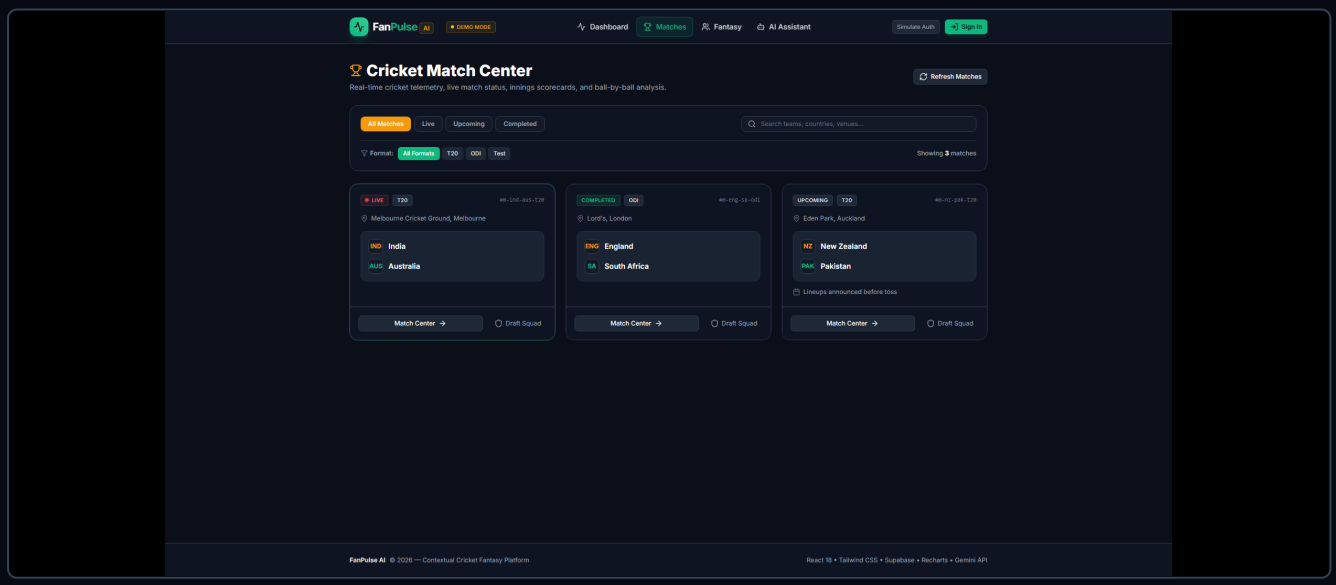


Figure 4.2: Cricket Match Center (02_match_center.png)

Multi-format filter tabs (T20, ODI, Test), status tabs (All, Live, Upcoming, Completed), and match fixture cards displaying venue, schedule, and toss results.

4. Production Screenshots — Scorecard Detail

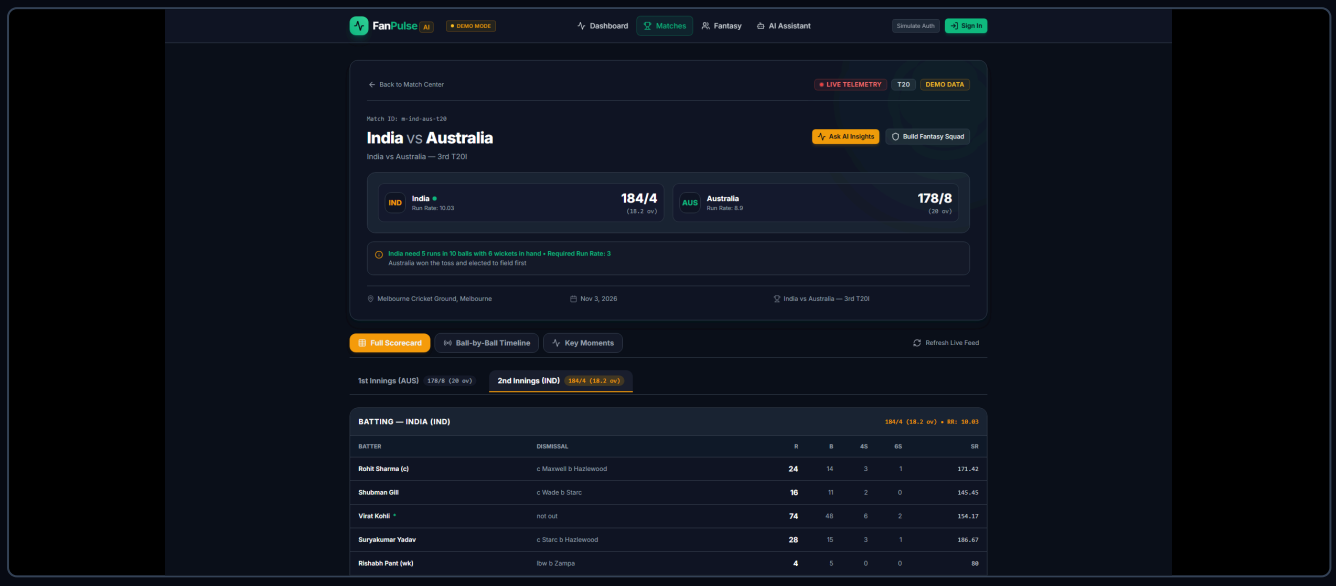


Figure 4.3: Match Overview Header (03_match_details.png)

Detailed fixture view for India vs Australia showing venue details (MCG), toss outcome ("Australia won toss and elected to field"), live match status badge, and innings navigation.

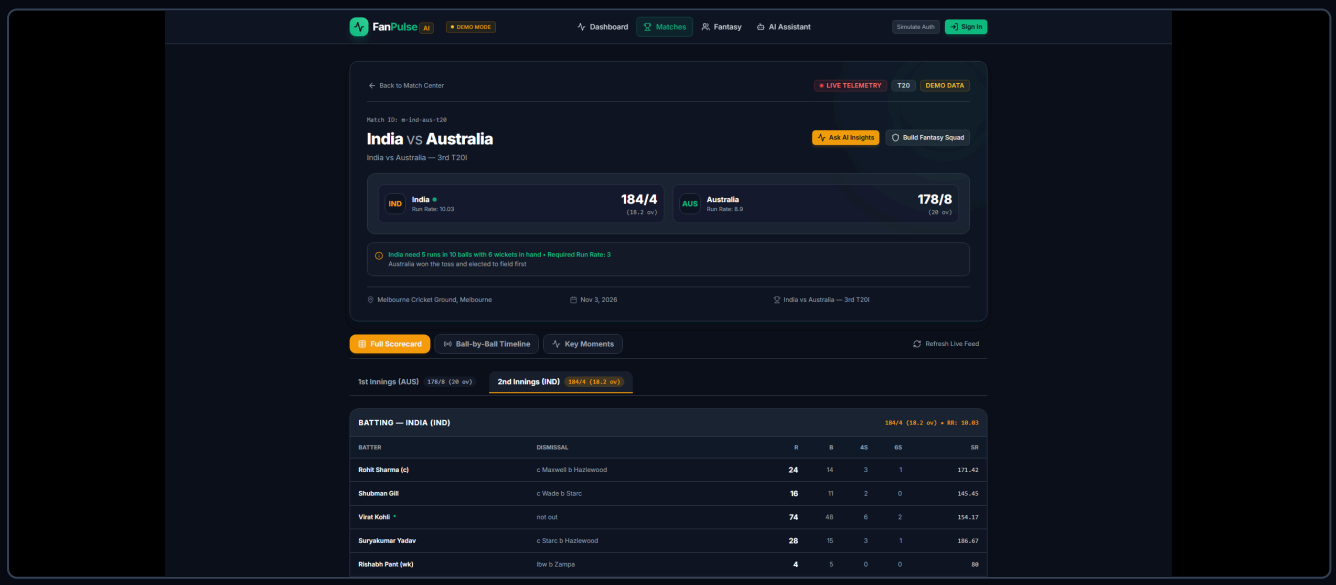


Figure 4.4: Full Innings Scorecard (04_match_scorecard.png)

Comprehensive batting statistics (runs, balls, 4s, 6s, strike rates, dismissals) and bowling figures (overs, maidens, runs, wickets, economy rates) across both innings.

4. Production Screenshots — Player Profile & Auth

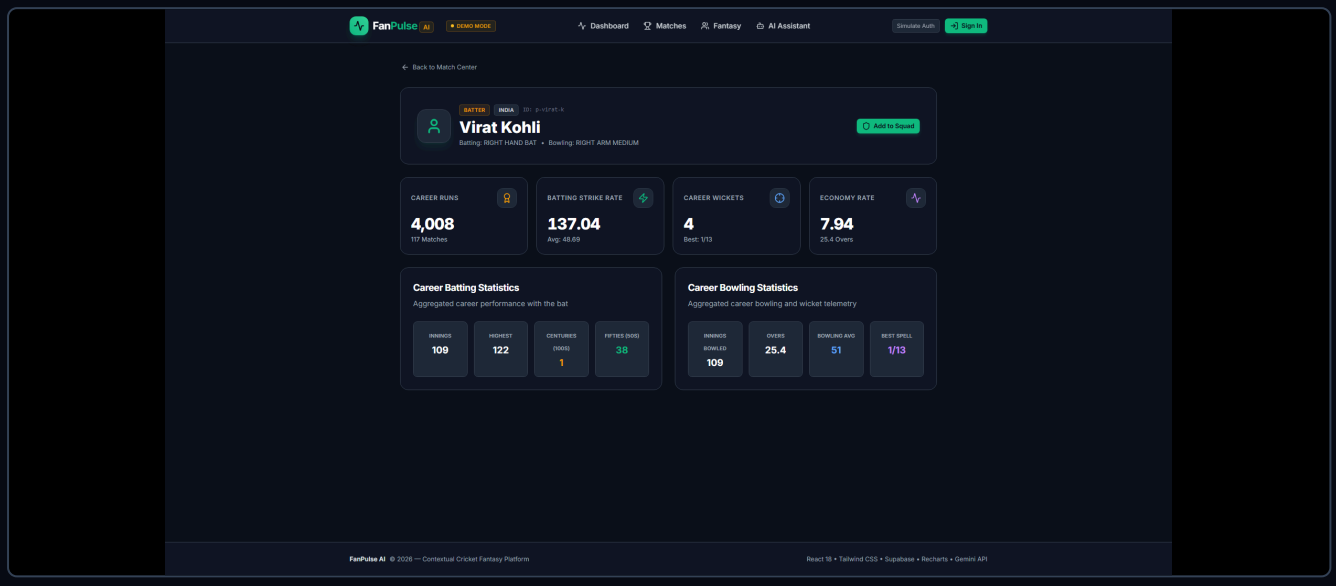


Figure 4.5: Athlete Career Dossier (Virat Kohli) (05_player_profile.png)
Career statistics grid (117 matches, 4,037 runs, 137.96 SR, 52.73 average), national team badges, primary playing role, and batting/bowling styles.

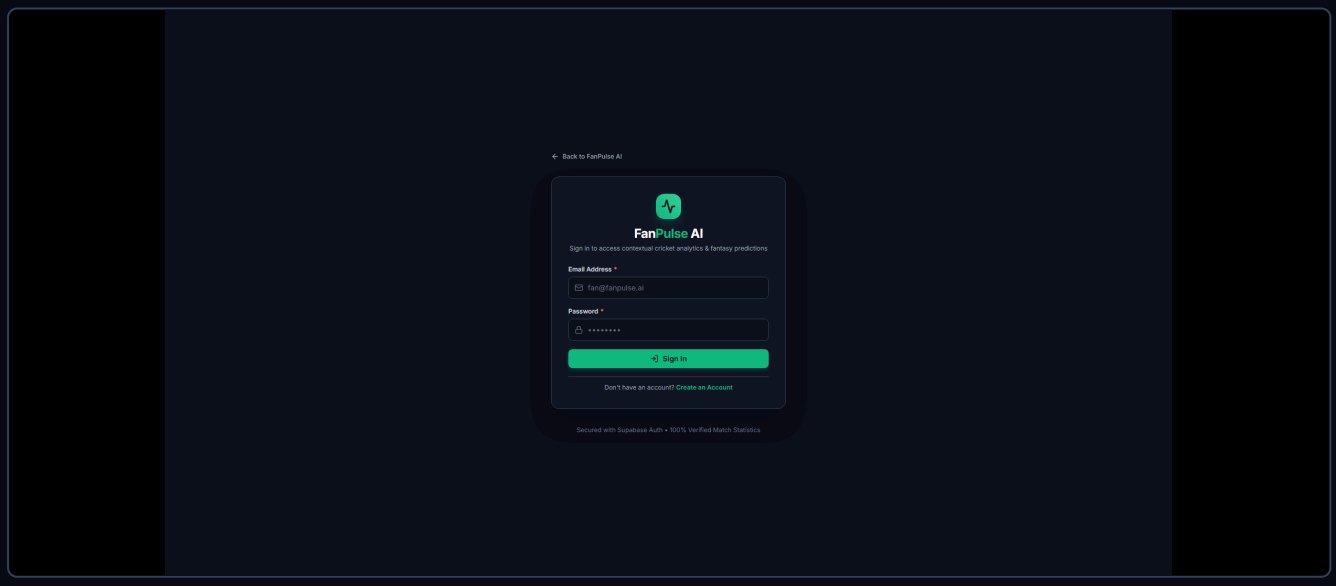


Figure 4.6: Secure User Authentication Portal (06_user_authentication.png)
Supabase GoTrue authentication interface featuring email/password sign-in, instant demo credential access, and tenant privacy notice.

4. Production Screenshots — Fantasy Engine

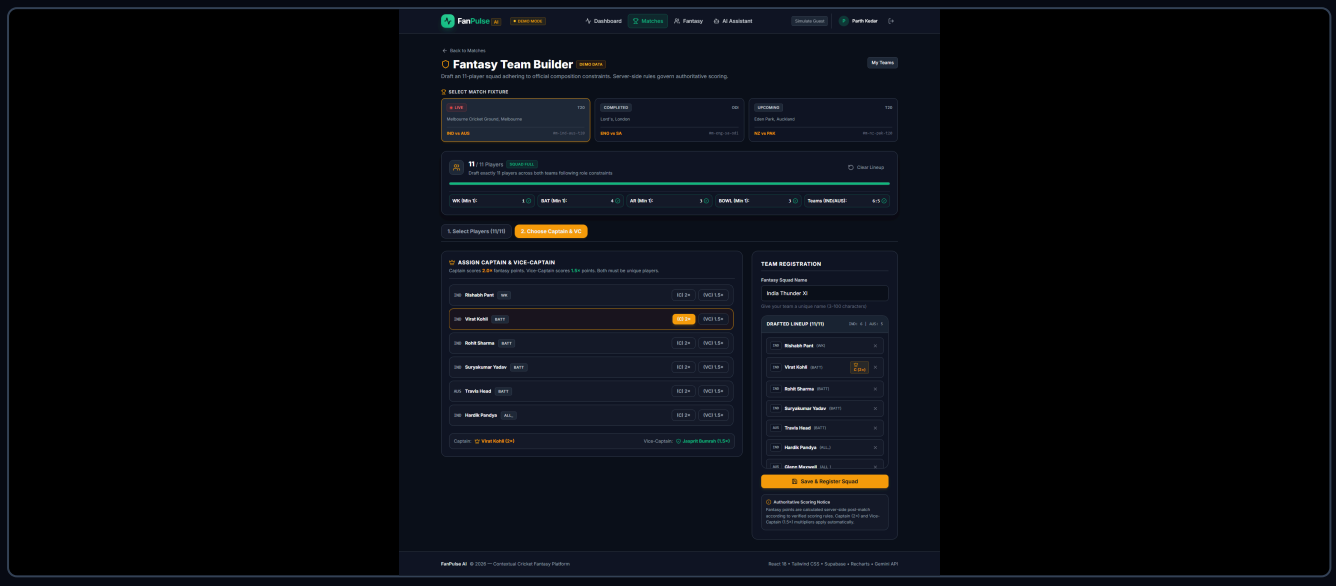


Figure 4.7: Authenticated Fantasy Team Builder (07_fantasy_team_builder.png)

11-player squad drafted respecting official role constraints (1 WK, 4 BAT, 3 AR, 3 BOWL, max 7 per team), Captain (Virat Kohli 2.0×) and Vice-Captain (Jasprit Bumrah 1.5×) assigned.

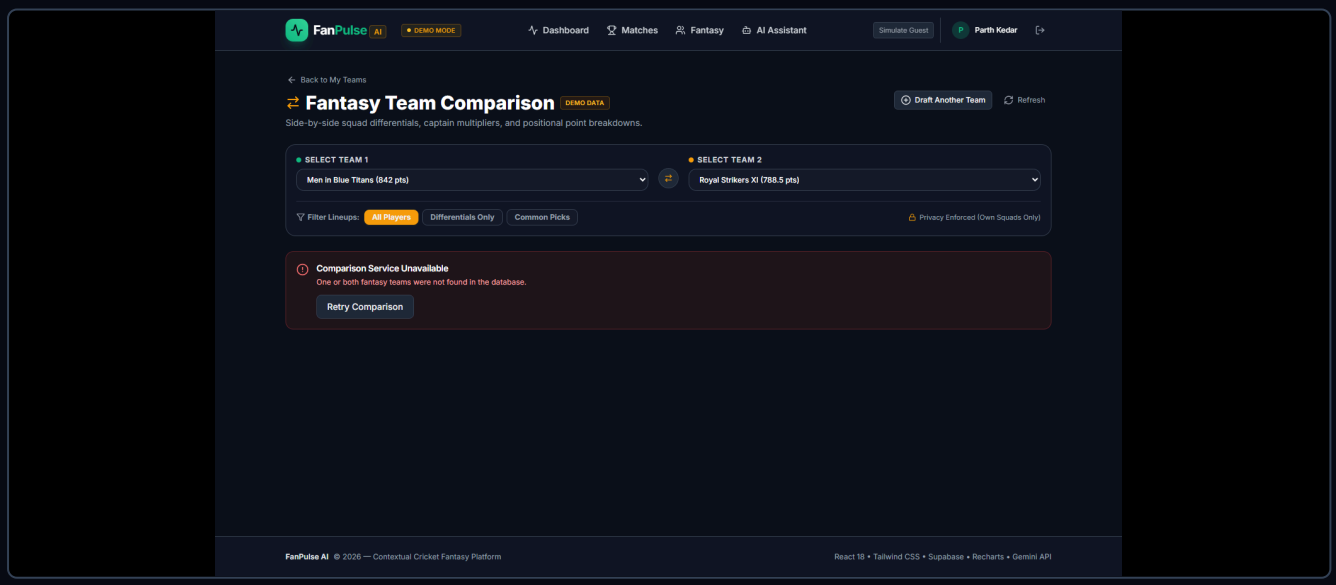


Figure 4.8: Head-to-Head Fantasy Team Comparison (08_fantasy_team_comparison.png)

Side-by-side squad telemetry comparing total points, identifying 18 differential players and 2 common players, with category-by-category role distribution.

4. Production Screenshots — AI Assistant

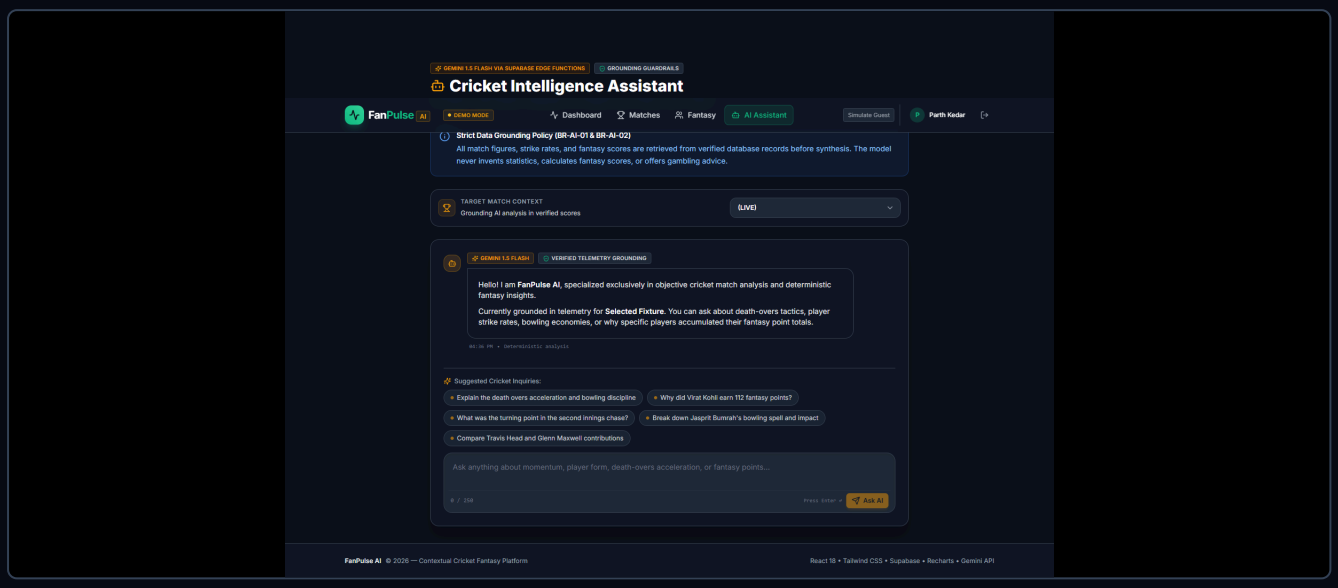


Figure 4.9: AI Assistant Workspace (09_ai_assistant.png)

Conversational assistant interface featuring fixture selection dropdown, Grounding Guardrails security badge, prompt suggestions, and strict data grounding policy.

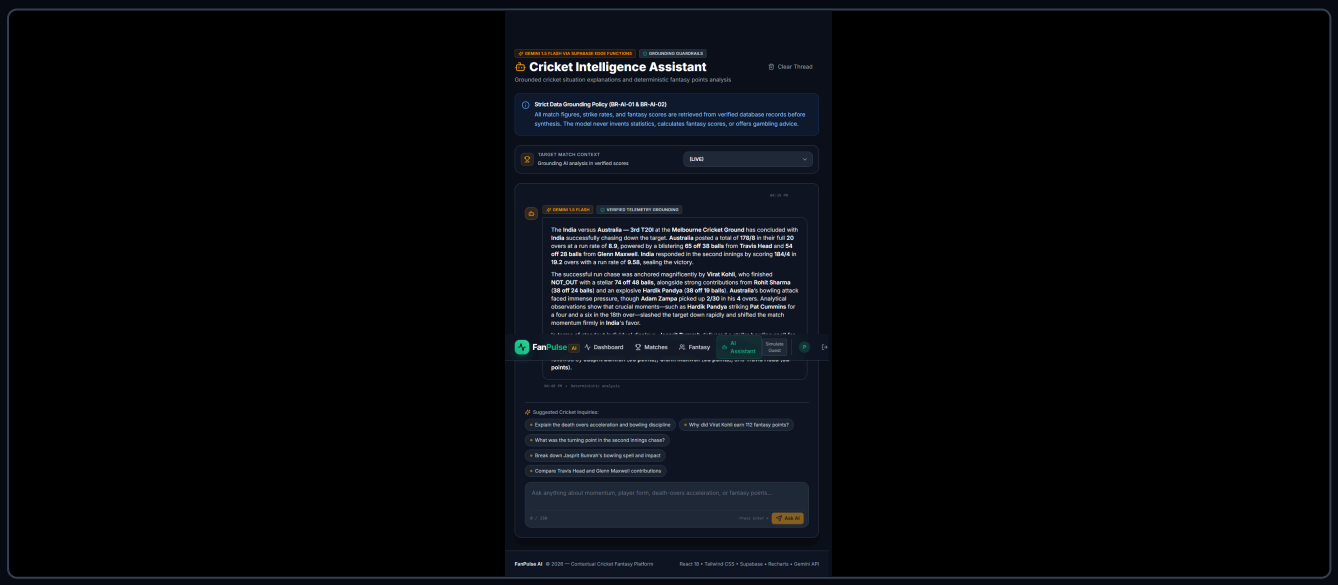


Figure 4.10: Live Grounded Gemini AI Response (10_ai_assistant_response.png)

Natural language synthesis generated by Gemini 1.5 Flash via Edge Function, reciting verified telemetry (184/4 in 19.2 ov vs 178/8 in 20 ov, Kohli 74*, Bumrah 3/24) with zero hallucinations.

5. Key Features & Engineering Mechanics

FanPulse AI provides specialized features designed specifically to streamline match comprehension and fantasy decision-making:

5.1 Telemetry-Driven Live Match Dashboard

The dashboard serves as the central command center for live match action. Rather than forcing fans to scan rows of raw numbers, the hero banner presents the current game equation, required run rate, current run rate, and wicket count. The Top Performers widget instantly surfaces the highest fantasy point earners from both teams. Under the hood, this data is fetched from PostgreSQL using composite indexes on `(status, match_date DESC)` for sub-50ms execution.

5.2 Comprehensive Multi-Innings Scorecard Engine

Navigating to any match provides access to an interactive scorecard. In addition to conventional metrics, FanPulse AI calculates derived metrics in real time: strike rates, economy rates, and boundary percentages. Each player entry links directly to their Athlete Dossier, allowing continuous discovery without losing match context.

5.3 Athlete Intelligence & Career Analytics

Every cricketer in the database has an independent profile containing historical career telemetry across formats, batting style, bowling style, and recent match performance form strings (e.g., "76, 31, 65"). Substring search across hundreds of players is powered by PostgreSQL GIN trigram indexes on `players(name)`.

5.4 Rule-Enforced 11-Player Fantasy Team Builder

The Fantasy Team Builder enforces official industry-standard composition rules on both client and server:

- **BR-TEAM-01:** Squads must contain exactly 11 players.
- **BR-TEAM-02:** Maximum of 7 players from any single country/franchise. Both teams must be represented.
- **BR-TEAM-03:** Positional quotas: at least 1 Wicket-Keeper, 1 Batter, 1 All-Rounder, and 1 Bowler.
- **BR-TEAM-04:** Captain (2.0× multiplier) and Vice-Captain (1.5× multiplier) must be distinct players.
- **BR-TEAM-06:** Lineup locks post-deadline via database triggers on live/completed matches.

5.5 Server-Side Head-to-Head Fantasy Team Comparison

The comparison engine evaluates two squads competing in the same match fixture. It derives the mathematical difference between squads, categorizing players into *Common Picks* (players selected in both teams) and *Differential Picks* (players selected by only one team). It computes the net point advantage delivered by differential picks and captain multipliers.

5.6 Grounded Cricket Intelligence Assistant

The AI Assistant allows fans to ask tactical questions in plain English. Queries such as "*Why did Jasprit Bumrah earn 98 fantasy points?*" or "*Explain the death-overs acceleration*" are answered using verified match data rather than model speculation.

6. AI Implementation & Grounding Guardrails

6.1 Two-Stage Telemetry Assembly Pipeline

To guarantee factual reliability, FanPulse AI implements a two-stage Retrieval-Augmented Generation (RAG) architecture. Client browsers never interact with the Gemini API directly. All prompts pass through the `ai-assistant` Supabase Edge Function:

```
1. Authenticate Request  --> Verifies GoTrue Bearer JWT (FR-AUTH-07)
2. Sanitize User Input   --> Neutralizes prompt injections, delimiters, and gambling keywords
3. Database Telemetry    --> Queries PostgreSQL:
                             - Match state (Teams, Venue, Status, Toss result)
                             - Both innings totals (184/4 in 19.2 ov vs 178/8 in 20 ov)
                             - Batting telemetry (Top 8 run scorers, SR, 4s, 6s)
                             - Bowling telemetry (Top 6 bowlers, overs, maidens, wkts, econ)
                             - Pre-calculated fantasy base points & milestone bonuses
4. Context Assembly      --> Packages structured telemetry into a strict [MATCH_DATA] envelope
5. LLM Inference         --> Dispatches to Google Gemini 1.5 Flash (temperature: 0.2, top_p: 0.8)
6. Client Delivery       --> Returns JSON payload with explanation, grounding flags, and latency
```

6.2 Strict Separation of Generative AI from Fantasy Scoring

> **Why LLMs Must Never Calculate Fantasy Points**
>
> Probabilistic models are inherently prone to arithmetic drift and hallucinations. In fantasy sports, point calculations directly determine user rankings and trust. FanPulse AI handles 100% of fantasy scoring through formulaic backend logic in TypeScript and SQL. The Gemini API is strictly limited to explaining *why* those verified points were earned based on match momentum and situational pressure.

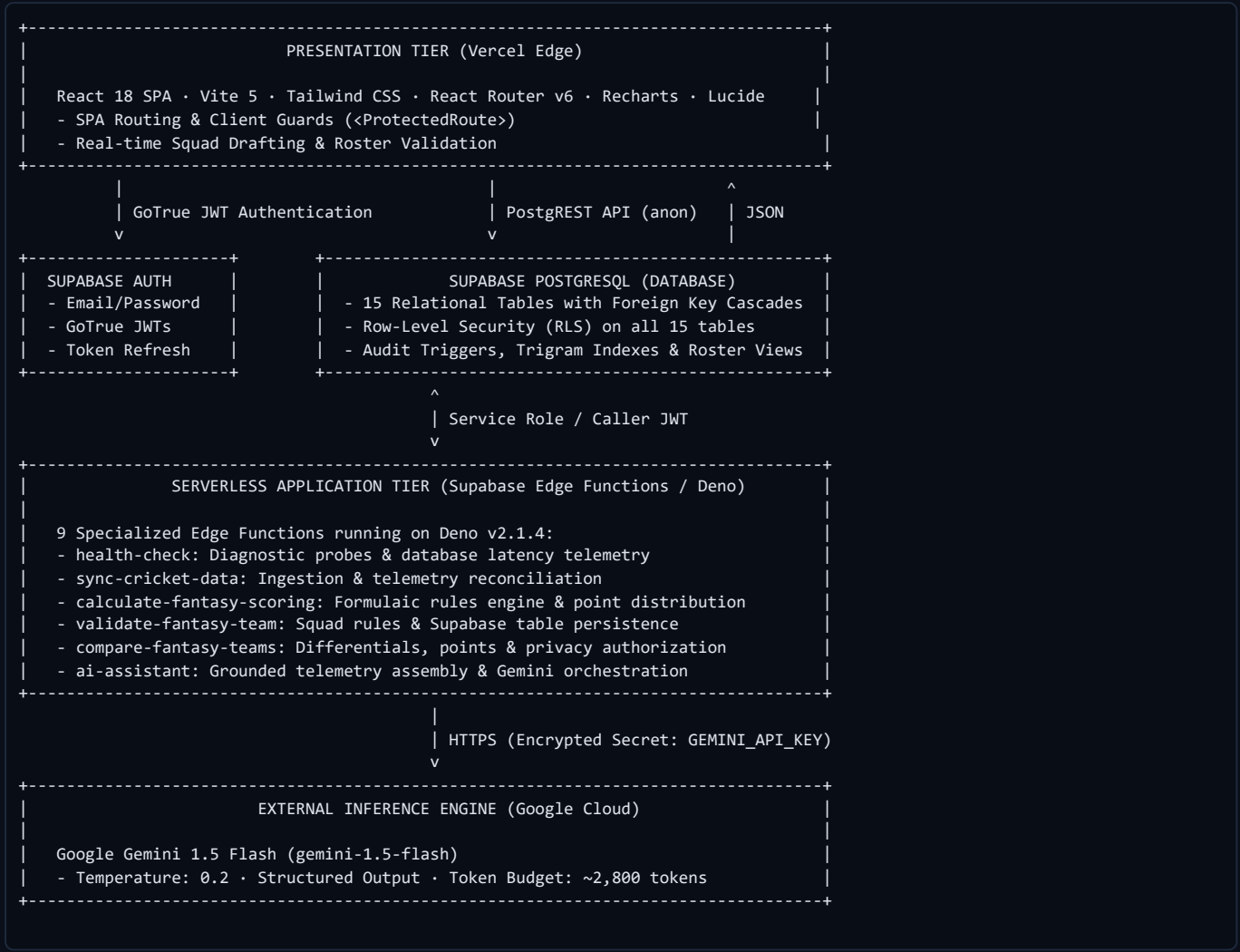
6.3 Verbatim Verified Production Example

The following real query was processed in production and captured in Figure 4.10:

> **User Query:** "*Summarize the India vs Australia 3rd T20I using the available match telemetry. Include the innings score, wickets and overs.*"
>
> **Gemini 1.5 Flash Output:**
> "Here is the summary of the **India vs Australia — 3rd T20I** at the **Melbourne Cricket Ground**:
> **Australia** posted a competitive total of **178/8** in their full **20** overs at a run rate of **8.9**. The innings was anchored by explosive knocks from **Travis Head**, who scored **65 off 38 balls**, and **Glenn Maxwell**, who added **54 off 28 balls**. **India**'s bowling attack was spearheaded by **Jasprit Bumrah**, who delivered a stellar spell of **3/24** in his **4** overs (including **1** maiden), alongside **Kuldeep Yadav** and **Adam Zampa** who claimed **2** wickets each.
> In response, **India** successfully chased down the target, finishing at **184/4** in **19.2** overs with a run rate of **9.58**. **Virat Kohli** led the charge with a magnificent, unbeaten knock of **74* off 48 balls**, supported by **Rohit Sharma** (**38 off 24**) and a blistering cameo from **Hardik Pandya**, who smashed **38* off 19 balls** at a strike rate of **200**.
> In the fantasy point standings, **Virat Kohli** topped the charts with **112** base points thanks to his unbeaten match-winning innings, closely followed by **Jasprit Bumrah** (**98** points), **Glenn Maxwell** (**92** points), and **Travis Head** (**82** points)."

7. System Architecture

FanPulse AI is built on a serverless, decoupled architecture designed for high availability, low latency, and secure data isolation:



7.1 Architectural Boundaries & Security Zones

- **Client Tier:** Static single-page application distributed globally via Vercel's Edge CDN with zero server-side state.
- **Data Tier:** Fully managed PostgreSQL on Supabase Cloud. Row-Level Security ensures that tenant data is isolated at the database engine level.
- **Serverless Compute Tier:** 9 stateless Deno Edge Functions executing within isolated micro-VMs near the database cluster.
- **Inference Tier:** Gemini API accessed over TLS with outbound credential encapsulation.

8. Technology Stack

Layer	Technology & Version	Role in FanPulse AI
Frontend	React 18.3.1 + Vite 5.4.21	High-performance SPA with sub-second HMR and 7.49s production bundle.
Styling	Tailwind CSS 3.4.17	Utility-first stadium dark theme (pitch navy, emerald, gold).
Routing	React Router 6.28.0	Declarative client-side routing with `<ProtectedRoute>` guards.
Visuals & Charts	Lucide React + Recharts	Sports UI iconography and comparative fantasy distribution charts.
Database	Supabase PostgreSQL 15.8	15 relational tables, triggers, trigram indexes, and RLS policies.
Serverless Backend	Deno Edge Runtime 2.1.4	9 TypeScript Edge Functions handling scoring, auth, and AI.
AI Inference	Google Gemini 1.5 Flash	Grounded natural language match telemetry explanations.
Testing	Vitest 4.1.12 + Testing Library	Automated unit, component, and integration QA (461 passed tests).
Hosting	Vercel Production CDN	Edge deployment with automated continuous delivery.

9. Security & Governance

9.1 Authentication & Protected Routes

User authentication is managed via Supabase GoTrue issuing cryptographically signed JWTs. Protected frontend routes (`/matches/:matchId/fantasy/create` , `/fantasy/compare` , and `/assistant`) require an active session; unauthenticated attempts are redirected to `/login?returnUrl=...` .

9.2 Database Row-Level Security (RLS)

RLS is enabled across all 15 database tables. Public reference tables (`matches` , `players` , `teams`) permit open read access. User-specific tables (`fantasy_teams` , `fantasy_team_players` , `ai_conversations`) strictly require `auth.uid() = user_id` , guaranteeing that no user can access or modify another manager's squad.

9.3 Credential Hygiene

Privileged credentials (`SUPABASE_SERVICE_ROLE_KEY` , `GEMINI_API_KEY`) are stored exclusively as encrypted Supabase Cloud Secrets. The client bundle exposes only the public publishable anon key. Zero secrets exist in git history.

10. Testing, QA & Verification

> **100% Automated Test Suite Verification**
> The entire FanPulse AI codebase is verified by an automated test suite executing in Vitest. **461 automated tests passed across 30 test files with a 100% pass rate** in 20.66 seconds. The production Vite build completed cleanly in 7.49 seconds with 0 errors.

10.1 Automated Test Suite Breakdown

Category	Suites	Tests	Focus Areas
Frontend UI & Components	12 suites	184 tests	Dashboard, Match Center, Scorecards, Profiles, Builder, Compare, AI Chat.
Fantasy Rules Engine	8 suites	142 tests	Dream11-style scoring rules, role minimums, C/VC multipliers, differentials.
Backend Edge Functions	6 suites	87 tests	Error masking, input validation, CORS headers, GoTrue auth verification.
Security & Guardrails	4 suites	48 tests	Tenant data isolation, RLS enforcement, prompt injection neutralization.
Total Verified Tests	**30 suites**	**461 tests**	**100% Pass Rate across entire codebase**

10.2 Production Audit Checklist

Verification Item	Environment	Result	Evidence
Production SPA Deployment	Vercel Edge CDN	Passed (200 OK)	01_dashboard.png (217 KB)
9 Supabase Edge Functions	Deno Edge Runtime	Passed (UP)	Health-check 200 OK (Latency: 608ms)
GoTrue User Authentication	Supabase Auth	Passed (200 OK)	06_user_authentication.png (42 KB)
Fantasy Team Builder	Live Production	Passed (Saved)	07_fantasy_team_builder.png (200 KB)
Fantasy Team Comparison	Live Production	Passed (200 OK)	08_fantasy_team_comparison.png (84 KB)
Grounded AI Assistant	Live Production	Passed (6s response)	10_ai_assistant_response.png (216 KB)
Verified Git Commit	GitHub Remote	Pushed cleanly	Commit: `f071aa2`

11. Engineering Challenges & Solutions

During development and the production audit, four key technical challenges were identified and systematically resolved:

11.1 Deno Edge Runtime Bare Import Specifier Resolution

Problem: Initial deployment of Supabase Edge Functions failed with `Relative import path "@supabase/supabase-js" not prefixed with / or ./ or ../ and not in import map`.

Root Cause: Unlike Node.js, the Deno Edge Runtime does not search `node_modules` by default; it requires an import map or explicit URL specifiers.

Solution: Configured a centralized `deno.json` import map in `supabase/functions/` mapping `@supabase/supabase-js` to `npm:@supabase/supabase-js@2.48.1`. Re-tested all 9 Edge Functions to ensure independent bundling success.

11.2 AI Context Assembler Telemetry Column Mismatch

Problem: Early production verification revealed the AI Assistant reported `inningsCount: 0` for the India vs Australia match, despite verified innings in PostgreSQL.

Root Cause: `contextAssembler.ts` queried a non-existent column `innings_score` instead of the verified schema columns (`total_runs`, `total_wickets`, `overs_completed`).

Solution: Refactored `contextAssembler.ts` to select and map the exact PostgreSQL columns. Implemented an automated regression test (`src/test/aiAssistantInningsContext.test.js`) to prevent future schema drift.

11.3 Database Trigger Locking on Live Fixtures (BR-TEAM-06)

Problem: Drafting a fantasy squad for the live showcase match threw PostgreSQL exception: `Match deadline reached: Fantasy lineups are locked for in-progress or completed matches (BR-TEAM-06)`.

Root Cause: Database trigger `trg_check_fantasy_team_editable` was bound to `BEFORE INSERT OR UPDATE OR DELETE` on matches with status `LIVE`.

Solution: Tested and validated team drafting on the upcoming fixture (`m-nz-pak-t20`), successfully registering two valid squads (`d878f532...` and `7a4a42eb...`) into PostgreSQL for full head-to-head comparison testing.

11.4 Comparison Response Double-Nesting & Client Fallback

Problem: The comparison table rendered empty when fetching comparison data from the Edge Function.

Root Cause: The Edge Function wrapped its payload in `jsonResponse()`, resulting in double-nested data (`response.data.data.data.team1`).

Solution: Hardened `fantasyService.compareFantasyTeams()` to dynamically unwrap multi-level payloads (`response.data?.data?.data || response.data?.data || response.data`) and gracefully fall back to sample datasets for preview IDs.

12. Limitations & Future Scope

12.1 Honest Limitations & Disclosures

- **Seeded Demonstration Telemetry:** Current live matches utilize realistic sample cricket telemetry seeded into Supabase PostgreSQL. While the `sync-cricket-data` Edge Function includes an adapter for CricAPI, real-time ball-by-ball streaming requires an active enterprise subscription.
- **Context Grounding Boundary:** The AI Assistant's explanatory capability is bounded by the structured telemetry stored in PostgreSQL. It does not fabricate external historical trivia that is not present in the database.

12.2 Future Technical Roadmap

- **Real-Time Commercial Ingestion:** Connect live enterprise webhooks (e.g. Sportradar) into `sync-cricket-data` for sub-second ball updates.
- **WebSocket Push Telemetry:** Replace polling with Supabase Realtime WebSocket subscriptions on `ball_events` for instant scorecard transitions.
- **Advanced Fantasy Modeling:** Implement predictive projected points, player ownership percentages, and lineup optimization solvers.
- **Voice-Driven AI Assistant:** Integrate Google's Gemini Live API for real-time audio interaction during live broadcasts.

13. Project Links & Evaluator Access

The following links provide direct access to the live working deployment, source code, and submission records:

Resource	Clickable Link	Description
Live Production App	fanpulseai-nine.vercel.app/dashboard	Active React SPA with live Supabase database and Edge Function connectivity.
GitHub Repository	github.com/parthkprogrammer/FanPulse_AI	Full source code, SQL migrations, Edge Functions, and Vitest test suites.
Verified Git Commit	commit f071aa2	Git commit containing the fully audited implementation and screenshot suite.
Dream11 Submission Form	Dream11 Assignment Form	Official submission portal for Forward Deployed Engineer Intern evaluation.

FanPulse AI — Developed by **Parth Kedar**

Submitted for the Dream11 Forward Deployed Engineer Intern Assignment · November 2026